Step 3 — The Pipeline — Solution

> Back to the step · Hint 1 · Hint 2

Story: `run_research` ties it together — plan, build the search, execute, compile the cited report — plus `main.py` and the integration test. **If not:** the three tested pieces never meet, and there's no single call that researches a question end to end.

Sample sources (`data/sources/`)

```
# data/sources/rag_overview.md
Retrieval-augmented generation (RAG) first retrieves relevant documents for a
question, then generates an answer grounded in them. It was introduced by
Lewis et al. at Meta AI in 2020.
```

```
# data/sources/chunking.md
Chunking splits a document into pieces before embedding. Where you split
matters: cut a fact in half and neither chunk can answer a question about it.
Recursive character splitting tries natural boundaries first.
```

```
# data/sources/agents.md
An AI agent is a model in a loop: it thinks, calls a tool, observes the result,
and repeats until it can answer. A step limit stops it from looping forever.
```

The pipeline

```
# src/pipeline.py
# Plan the question, build the search over the sources, run the plan, and
# compile the cited report.
from src.executor import execute_plan
from src.planner import make_plan
from src.rag.search import make_search
from src.rag.source_store import build_source_store
from src.report import compile_report
from src.utils.logger import get_logger
```

```
def run_research(model, question):
    logger = get_logger(__name__)
    plan = make_plan(model, question).sub_questions
    logger.info(f"planned {len(plan)} sub-question(s)")

    collection = build_source_store()
    search = make_search(collection)

    findings = execute_plan(plan, search)
    return compile_report(question, findings)
```

main.py

```
# main.py
import sys

from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_research
from src.report import format_report


def main():
    if len(sys.argv) < 2:
        print('usage: python main.py "your research question"')
        return
    question = sys.argv[1]

    config = load_config()
    model = get_model(config)
    report = run_research(model, question)
    print()
    print(format_report(report))


if __name__ == "__main__":
    main()
```

The integration test (real model + RAG)

```
# tests/integration/test_live_research.py
# Real model + RAG. Run from the repo root:
```

```
#   python -m tests.integration.test_live_research
from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_research
from src.report import format_report


def test_report_has_cited_findings():
    config = load_config()
    model = get_model(config)
    report = run_research(
        model, "Give me a briefing on retrieval-augmented generation")
    text = format_report(report)
    ok = len(report.findings) >= 2 and "[1]" in text and "Sources:" in text
    print(f"case 1 (several cited findings) -> {'OK' if ok else 'WRONG'}")


def test_sources_are_real_files():
    config = load_config()
    model = get_model(config)
    report = run_research(model, "What is RAG and how does chunking affect it?")
    real = True
    for finding in report.findings:
        if not finding.source.endswith(".md"):
            real = False
    print(f"case 2 (sources are real files) -> {'OK' if real else 'WRONG'}")


if __name__ == "__main__":
    test_report_has_cited_findings()
    test_sources_are_real_files()
```

Expected run:

```
$ python main.py "Give me a briefing on retrieval-augmented generation"

Research: Give me a briefing on retrieval-augmented generation

- What is RAG?
  RAG retrieves relevant documents, then generates an answer from them. [1]
- Who introduced RAG?
  RAG ... was introduced by Lewis et al. at Meta AI in 2020. [1]
- How does chunking affect RAG?
  Chunking splits a document into pieces before embedding... [2]

Sources:
```

```
  [1] rag_overview.md
  [2] chunking.md
```

That's the whole project: the question planned into sub-questions, each researched against your documents, and the findings compiled into a report where every claim is cited. Plan first, execute each step, compile — and the citations are what make the answer something a reader can actually trust.